

Punto 3.11 - Composición de consultas y ejecución diferida

Práctica

Ejercicio: Añadir métodos de composición de consultas al repositorio de órdenes del proyecto AceriaData. Crear métodos que construyan consultas con filtros, ordenaciones y paginación de forma incremental. Crear un caso de uso que ejecute estas consultas y analice el SQL generado.

Contexto del proyecto: En el punto 3.10 se exploró la carga Explicit, incluyendo Entry().Reference().Load(), Entry().Collection().Load(), IsLoaded y Query(). En este punto se profundiza en la composición de consultas y la ejecución diferida. Esta técnica se usará en el punto 3.12 para las buenas prácticas y en el Módulo 4 para las optimizaciones de rendimiento.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Crear el DTO FiltroOrdenesDto

Crear el archivo src/AceriaData.Application/Dtos/FiltroOrdenesDto.cs:

```
namespace AceriaData.Application.Dtos;

public class FiltroOrdenesDto
{
    public string? Cliente { get; set; }
    public string? Estado { get; set; }
    public DateTime? FechaDesde { get; set; }
    public DateTime? FechaHasta { get; set; }
    public string? OrdenarPor { get; set; }
    public bool Descendente { get; set; }
    public int? Pagina { get; set; }
    public int? TamanoPagina { get; set; }
}
```

Línea 1: namespace AceriaData.Application.Dtos; → declara el espacio de nombres.

Línea 3: public class FiltroOrdenesDto → declara el DTO de filtros.

Línea 5: public string? Cliente { get; set; } → filtro por cliente.

Línea 6: public string? Estado { get; set; } → filtro por estado.

Línea 7: public DateTime? FechaDesde { get; set; } → filtro por fecha desde.

Línea 8: public DateTime? FechaHasta { get; set; } → filtro por fecha hasta.

Línea 9: public string? OrdenarPor { get; set; } → criterio de ordenación.

Línea 10: public bool Descendente { get; set; } → indica si la ordenación es descendente.

Línea 11: public int? Pagina { get; set; } → número de página.

Línea 12: public int? TamanoPagina { get; set; } → tamaño de la página.

Error común: si los filtros son de tipo string no anulable, el DTO requiere que se asignen siempre. Se deben declarar como string? para que sean opcionales.

| Paso 3: Añadir el método de composición a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerConFiltros(FiltroOrdenesDto filtro);
    int ContarConFiltros(FiltroOrdenesDto filtro);

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerConFiltros(FiltroOrdenesDto filtro); → método que obtiene las órdenes con filtros.

Línea 11: int ContarConFiltros(FiltroOrdenesDto filtro); → método que cuenta las órdenes con filtros.

Línea 13: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 14: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

Error común: si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 4: Implementar el método de composición en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs:

```

public List<OrdenFabricacion> ObtenerConFiltros(FiltroOrdenesDto filtro)
{
    var consulta = ConstruirConsultaConFiltros(filtro);
    return consulta.ToList();
}

public int ContarConFiltros(FiltroOrdenesDto filtro)
{
    var consulta = ConstruirConsultaConFiltros(filtro);
    return consulta.Count();
}

private IQueryable<OrdenFabricacion> ConstruirConsultaConFiltros(FiltroOrdenesDto filtro)
{
    var consulta = _context.OrdenesFabricacion.AsQueryable();

    if (!string.IsNullOrEmpty(filtro.Cliente))
    {
        consulta = consulta.Where(o => o.Cliente == filtro.Cliente);
    }

    if (!string.IsNullOrEmpty(filtro.Estado))
    {
        consulta = consulta.Where(o => o.Estado == filtro.Estado);
    }

    if (filtro.FechaDesde.HasValue)
    {
        consulta = consulta.Where(o => o.FechaCreacion >= filtro.FechaDesde.Value);
    }

    if (filtro.FechaHasta.HasValue)
    {
        consulta = consulta.Where(o => o.FechaCreacion <= filtro.FechaHasta.Value);
    }

    if (!string.IsNullOrEmpty(filtro.OrdenarPor))
    {
        consulta = filtro.OrdenarPor switch
        {
            "Cliente" => filtro.Descendente ? consulta.OrderByDescending(o => o.Cliente) :
            consulta.OrderBy(o => o.Cliente),
            "Estado" => filtro.Descendente ? consulta.OrderByDescending(o => o.Estado) :
            consulta.OrderBy(o => o.Estado),
            "FechaCreacion" => filtro.Descendente ? consulta.OrderByDescending(o => o.FechaCreacion) :
            consulta.OrderBy(o => o.FechaCreacion),
            "NumeroOrden" => filtro.Descendente ? consulta.OrderByDescending(o => o.NumeroOrden) :
            consulta.OrderBy(o => o.NumeroOrden),
            _ => consulta.OrderBy(o => o.NumeroOrden)
        };
    }
    else
    {
        consulta = consulta.OrderBy(o => o.NumeroOrden);
    }

    if (filtro.Pagina.HasValue && filtro.TamanoPagina.HasValue &&
        filtro.Pagina.Value > 0 && filtro.TamanoPagina.Value > 0)
    {
        consulta = consulta
            .Skip((filtro.Pagina.Value - 1) * filtro.TamanoPagina.Value)
            .Take(filtro.TamanoPagina.Value);
    }

    return consulta;
}

```

Línea 1: `public List ObtenerConFiltros(FiltroOrdenesDto filtro)` → declara el método que obtiene las órdenes con filtros.

Línea 3: `var consulta = ConstruirConsultaConFiltros(filtro);` → construye la consulta.

Línea 4: `return consulta.ToList();` → materializa la consulta.

Línea 7: `public int ContarConFiltros(FiltroOrdenesDto filtro)` → declara el método que cuenta las órdenes con filtros.

Línea 9: `var consulta = ConstruirConsultaConFiltros(filtro);` → construye la consulta.

Línea 10: `return consulta.Count();` → cuenta los elementos.

Línea 13: `private IQueryable ConstruirConsultaConFiltros(FiltroOrdenesDto filtro)` → declara el método que construye la consulta.

Línea 15: `var consulta = _context.OrdenesFabricacion.AsQueryable();` → inicia la consulta base.

Línea 17: `if (!string.IsNullOrEmpty(filtro.Cliente))` → comprueba si el filtro por cliente está presente.

Línea 19: `consulta = consulta.Where(o => o.Cliente == filtro.Cliente);` → añade el filtro por cliente.

Línea 22: `if (!string.IsNullOrEmpty(filtro.Estado))` → comprueba si el filtro por estado está presente.

Línea 24: `consulta = consulta.Where(o => o.Estado == filtro.Estado);` → añade el filtro por estado.

Línea 27: `if (filtro.FechaDesde.HasValue)` → comprueba si el filtro por fecha desde está presente.

Línea 29: `consulta = consulta.Where(o => o.FechaCreacion >= filtro.FechaDesde.Value);` → añade el filtro por fecha desde.

Línea 32: `if (filtro.FechaHasta.HasValue)` → comprueba si el filtro por fecha hasta está presente.

Línea 34: `consulta = consulta.Where(o => o.FechaCreacion <= filtro.FechaHasta.Value);` → añade el filtro por fecha hasta.

Línea 37: `if (!string.IsNullOrEmpty(filtro.OrdenarPor))` → comprueba si hay criterio de ordenación.

Línea 39: `consulta = filtro.OrdenarPor switch` → selecciona el criterio de ordenación.

Línea 41: `"Cliente" => filtro.Descendente ? consulta.OrderByDescending(o => o.Cliente) : consulta.OrderBy(o => o.Cliente),` → ordena por cliente.

Línea 42: `"Estado" => filtro.Descendente ? consulta.OrderByDescending(o => o.Estado) : consulta.OrderBy(o => o.Estado),` → ordena por estado.

Línea 43: `"FechaCreacion" => filtro.Descendente ? consulta.OrderByDescending(o => o.FechaCreacion) : consulta.OrderBy(o => o.FechaCreacion),` → ordena por fecha de creación.

Línea 44: `"NumeroOrden" => filtro.Descendente ? consulta.OrderByDescending(o => o.NumeroOrden) : consulta.OrderBy(o => o.NumeroOrden),` → ordena por número de orden.

Línea 45: `_ => consulta.OrderBy(o => o.NumeroOrden)` → ordenación por defecto.

Línea 46: `};` → cierra el switch.

Línea 49: `consulta = consulta.OrderBy(o => o.NumeroOrden);` → ordenación por defecto si no hay criterio.

Línea 52: `if (filtro.Pagina.HasValue && filtro.TamanoPagina.HasValue &&` → comprueba si hay paginación.

Línea 53: `filtro.Pagina.Value > 0 && filtro.TamanoPagina.Value > 0)` → comprueba que los valores son positivos.

Línea 55: `consulta = consulta` → inicia la paginación.

Línea 56: `.Skip((filtro.Pagina.Value - 1) * filtro.TamanoPagina.Value)` → salta las páginas anteriores.

Línea 57: `.Take(filtro.TamanoPagina.Value);` → toma el tamaño de la página.

Línea 60: `return consulta;` → devuelve la consulta construida.

Error común: si se llama a `ToList` dentro del método `ConstruirConsultaConFiltros`, la consulta se materializa antes de tiempo y se pierde la ventaja de la composición. Se debe devolver `IQueryable` y materializar solo al final.

| Paso 5: Crear el caso de uso de composición de consultas

Crear el archivo `src/AceriaData.Application/UseCases/ComposicionConsultasUseCase.cs`:

```
using AceriaData.Application.Dtos;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ComposicionConsultasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ComposicionConsultasUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== COMPOSICIÓN DE CONSULTAS ===");

        DemostrarFiltroPorCliente();
        DemostrarFiltroPorEstado();
        DemostrarFiltroPorRangoDeFechas();
        DemostrarFiltrosCombinados();
        DemostrarOrdenacionYPaginacion();
    }

    private void DemostrarFiltroPorCliente()
    {
        Console.WriteLine("\n--- Filtro por cliente ---");

        var filtro = new FiltroOrdenesDto { Cliente = "Constructora del Norte" };
        var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        }
    }

    private void DemostrarFiltroPorEstado()
    {
        Console.WriteLine("\n--- Filtro por estado ---");

        var filtro = new FiltroOrdenesDto { Estado = "Pendiente" };
        var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
        }
    }

    private void DemostrarFiltroPorRangoDeFechas()
    {
        Console.WriteLine("\n--- Filtro por rango de fechas ---");

        var filtro = new FiltroOrdenesDto
        {
            FechaDesde = new DateTime(2024, 1, 1),
            FechaHasta = new DateTime(2024, 6, 30)
        };
        var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);
    }
}
```

```

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarFiltrosCombinados()
    {
        Console.WriteLine("\n--- Filtros combinados ---");

        var filtro = new FiltroOrdenesDto
        {
            Cliente = "Constructora del Norte",
            Estado = "Pendiente",
            FechaDesde = new DateTime(2024, 1, 1)
        };
        var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");
        }
    }

    private void DemostrarOrdenacionYPaginacion()
    {
        Console.WriteLine("\n--- Ordenación y paginación ---");

        var filtro = new FiltroOrdenesDto
        {
            OrdenarPor = "FechaCreacion",
            Descendente = true,
            Pagina = 1,
            TamanoPagina = 2
        };
        var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);
        var total = _unidad.Ordenes.ContarConFiltros(filtro);

        Console.WriteLine($"Total de órdenes: {total} | Página 1 con 2 elementos: {ordenes.Count}");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($" {orden.NumeroOrden} | {orden.FechaCreacion:dd/MM/yyyy}");
        }
    }
}

```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class ComposicionConsultasUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public ComposicionConsultasUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== COMPOSICIÓN DE CONSULTAS ==="); → muestra la cabecera.

Línea 19: DemostrarFiltroPorCliente(); → llama al método de filtro por cliente.

Línea 20: DemostrarFiltroPorEstado(); → llama al método de filtro por estado.

Línea 21: `MostrarFiltroPorRangoDeFechas();` → llama al método de filtro por rango de fechas.

Línea 22: `MostrarFiltrosCombinados();` → llama al método de filtros combinados.

Línea 23: `MostrarOrdenacionYPaginacion();` → llama al método de ordenación y paginación.

Línea 26: `private void MostrarFiltroPorCliente()` → declara el método.

Línea 28: `Console.WriteLine("\n--- Filtro por cliente ---");` → muestra la cabecera.

Línea 30: `var filtro = new FiltroOrdenesDto { Cliente = "Constructora del Norte" };` → crea el filtro.

Línea 31: `var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);` → llama al método del repositorio.

Línea 33: `Console.WriteLine($"Órdenes: {ordenes.Count}");` → muestra el número de órdenes.

Línea 34: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 36: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");` → muestra los datos.

Línea 39: `private void MostrarFiltroPorEstado()` → declara el método.

Línea 41: `Console.WriteLine("\n--- Filtro por estado ---");` → muestra la cabecera.

Línea 43: `var filtro = new FiltroOrdenesDto { Estado = "Pendiente" };` → crea el filtro.

Línea 44: `var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);` → llama al método del repositorio.

Línea 46: `Console.WriteLine($"Órdenes: {ordenes.Count}");` → muestra el número de órdenes.

Línea 47: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 49: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");` → muestra los datos.

Línea 52: `private void MostrarFiltroPorRangoDeFechas()` → declara el método.

Línea 54: `Console.WriteLine("\n--- Filtro por rango de fechas ---");` → muestra la cabecera.

Línea 56: `var filtro = new FiltroOrdenesDto` → crea el filtro.

Línea 58: `FechaDesde = new DateTime(2024, 1, 1),` → asigna la fecha desde.

Línea 59: `FechaHasta = new DateTime(2024, 6, 30)` → asigna la fecha hasta.

Línea 60: `};` → cierra el filtro.

Línea 61: `var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);` → llama al método del repositorio.

Línea 63: `Console.WriteLine($"Órdenes: {ordenes.Count}");` → muestra el número de órdenes.

Línea 64: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 66: `Console.WriteLine($" {orden.NumeroOrden} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 69: `private void MostrarFiltrosCombinados()` → declara el método.

Línea 71: `Console.WriteLine("\n--- Filtros combinados ---");` → muestra la cabecera.

Línea 73: `var filtro = new FiltroOrdenesDto` → crea el filtro.

Línea 75: `Cliente = "Constructora del Norte",` → asigna el cliente.

Línea 76: `Estado = "Pendiente",` → asigna el estado.

Línea 77: `FechaDesde = new DateTime(2024, 1, 1)` → asigna la fecha desde.

Línea 78: `};` → cierra el filtro.

Línea 79: `var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);` → llama al método del repositorio.

Línea 81: `Console.WriteLine($"Órdenes: {ordenes.Count}");` → muestra el número de órdenes.

Línea 82: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 84: `Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Línea 87: `private void DemostrarOrdenacionYPaginacion()` → declara el método.

Línea 89: `Console.WriteLine("\n--- Ordenación y paginación ---");` → muestra la cabecera.

Línea 91: `var filtro = new FiltroOrdenesDto` → crea el filtro.

Línea 93: `OrdenarPor = "FechaCreacion",` → asigna el criterio de ordenación.

Línea 94: `Descendente = true,` → indica ordenación descendente.

Línea 95: `Pagina = 1,` → asigna la página.

Línea 96: `TamanoPagina = 2` → asigna el tamaño de la página.

Línea 97: `};` → cierra el filtro.

Línea 98: `var ordenes = _unidad.Ordenes.ObtenerConFiltros(filtro);` → llama al método del repositorio.

Línea 99: `var total = _unidad.Ordenes.ContarConFiltros(filtro);` → cuenta las órdenes con el filtro.

Línea 101: `Console.WriteLine($"Total de órdenes: {total} | Página 1 con 2 elementos: {ordenes.Count}");` → muestra el total y el número de órdenes.

Línea 102: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 104: `Console.WriteLine($" {orden.NumeroOrden} | {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos.

Error común: si el método `ConstruirConsultaConFiltros` materializa la consulta con `ToList`, la composición se pierde. Se debe devolver `IQueryable` y materializar solo al final.

Paso 6: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ComposicionConsultasUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

Paso 7: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ComposicionConsultasUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

Paso 8: Insertar datos de prueba con varias órdenes

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes de prueba:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Norte", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Este", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) }
    };

    context OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base.

Línea 5: context.Database.EnsureCreated(); → crea la base.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... }, → quinta orden.

Línea 14: new OrdenFabricacion { ... } → sexta orden.

Línea 15: }; → cierra la lista.

Línea 17: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 18: context.SaveChanges(); → inserta las órdenes.

Error común: si se insertan las órdenes con el mismo número, la clave alternativa rechaza la inserción.

Paso 9: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada filtro. Se observan los filtros por cliente, por estado, por rango de fechas, combinados y con ordenación y paginación.

Paso 10: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== COMPOSICIÓN DE CONSULTAS ===
```

```
--- Filtro por cliente ---
```

```
Órdenes: 3
```

```
OF-2024-0001 | Constructora del Norte | Pendiente
OF-2024-0003 | Constructora del Norte | EnProceso
OF-2024-0005 | Constructora del Norte | Pendiente
```

```
--- Filtro por estado ---
```

```
Órdenes: 4
```

```
OF-2024-0001 | Constructora del Norte | Pendiente
OF-2024-0002 | Constructora del Sur | Pendiente
OF-2024-0004 | Constructora del Este | Pendiente
OF-2024-0005 | Constructora del Norte | Pendiente
```

```
--- Filtro por rango de fechas ---
```

```
Órdenes: 5
```

```
OF-2024-0001 | 15/01/2024
OF-2024-0002 | 20/02/2024
OF-2024-0003 | 10/03/2024
OF-2024-0004 | 05/04/2024
OF-2024-0005 | 12/05/2024
```

```
--- Filtros combinados ---
```

```
Órdenes: 2
```

```
OF-2024-0001 | Constructora del Norte | Pendiente | 15/01/2024
OF-2024-0005 | Constructora del Norte | Pendiente | 12/05/2024
```

```
--- Ordenación y paginación ---
```

```
Total de órdenes: 6 | Página 1 con 2 elementos: 2
```

```
OF-2024-0006 | 18/06/2024
OF-2024-0005 | 12/05/2024
```

La primera sección muestra el filtro por cliente. La segunda sección muestra el filtro por estado. La tercera sección muestra el filtro por rango de fechas. La cuarta sección muestra los filtros combinados. La quinta sección muestra la ordenación y paginación.

Observaciones: cada consulta se ha construido de forma incremental con los filtros aplicados. La consulta de ordenación y paginación incluye Skip y Take, que se traducen a OFFSET y FETCH en SQL Server.

Paso 11: Diagnosticar un error común

Modificar el método ObtenerConFiltros para materializar la consulta antes de aplicar todos los filtros:

```
csharp
public List<OrdenFabricacion> ObtenerConFiltros(FiltroOrdenesDto filtro)
{
    var consulta = _context.OrdenesFabricacion.ToList();

    if (!string.IsNullOrEmpty(filtro.Cliente))
    {
        consulta = consulta.Where(o => o.Cliente == filtro.Cliente).ToList();
    }

    if (!string.IsNullOrEmpty(filtro.Estado))
    {
        consulta = consulta.Where(o => o.Estado == filtro.Estado).ToList();
    }

    return consulta;
}
```

Resultado esperado: el código carga todas las órdenes en memoria y después filtra. Si la tabla tiene un millón de filas, se transfieren todas a memoria antes de filtrar. Esto degrada el rendimiento.

Solución: construir la consulta con IQueryable y materializar solo al final.

```
public List<OrdenFabricacion> ObtenerConFiltros(FiltroOrdenesDto filtro)
{
    var consulta = ConstruirConsultaConFiltros(filtro);
    return consulta.ToList();
}
```

Resultado esperado con la solución: la consulta se construye de forma incremental y se materializa solo al final. Los filtros se aplican en SQL.

Errores comunes del ejercicio

Error	Causa	Solución
Materialización prematura	Se llamó a ToList antes de aplicar todos los filtros	
Materializar solo al final		
Filtros en memoria	Se materializó la consulta antes de filtrar	Construir la consulta con IQueryable
Múltiples consultas	Se materializó varias veces	Materializar una sola vez al final
ThenBy sin OrderBy	Se llamó a ThenBy sin OrderBy previo	Llamar a OrderBy primero
Skip sin OrderBy	Se aplicó Skip sin ordenar	Aplicar OrderBy antes de Skip
Paginación incorrecta	Se calculó mal el Skip	Usar (pagina - 1) * tamañoPagina

Reto resuelto: Consulta compuesta con filtros dinámicos

Reto: Crear un método en el repositorio que construya una consulta con filtros dinámicos basados en una lista de predicados. Añadir el método a la interfaz, la implementación y una demostración en el caso de uso.

Solución paso a paso:

Paso 1: Añadir el método a la interfaz IOrdenRepositorio:

```
csharp
List<OrdenFabricacion>
ObtenerConPredicados(List<System.Linq.Expressions.Expression<Func<OrdenFabricacion, bool>>> predicados);
```

Línea 1: List ObtenerConPredicados(List<System.Linq.Expressions.Expression<Func<OrdenFabricacion, bool>>> predicados); → declara el método que acepta una lista de predicados.

Paso 2: Implementar el método en OrdenRepositorio:

```
public List<OrdenFabricacion>
ObtenerConPredicados(List<System.Linq.Expressions.Expression<Func<OrdenFabricacion, bool>>> predicados)
{
    var consulta = _context.OrdenesFabricacion.AsQueryable();

    foreach (var predicado in predicados)
    {
        consulta = consulta.Where(predicado);
    }

    return consulta.OrderBy(o => o.NumeroOrden).ToList();
}
```

Línea 1: public List

ObtenerConPredicados(List<System.Linq.Expressions.Expression<Func<OrdenFabricacion, bool>>> predicados) → declara el método.

Línea 3: var consulta = _context.OrdenesFabricacion.AsQueryable(); → inicia la consulta base.

Línea 5: foreach (var predicado in predicados) → itera sobre los predicados.

Línea 7: consulta = consulta.Where(predicado); → aplica cada predicado.

Línea 10: return consulta.OrderBy(o => o.NumeroOrden).ToList(); → ordena y materializa la consulta.

Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarPredicadosDinamicos()
{
    Console.WriteLine("\n--- Predicados dinámicos ---");

    var predicados = new
    List<System.Linq.Expressions.Expression<Func<AceriaData.Domain.Entities.OrdenFabricacion, bool>>>
    {
        o => o.Estado == "Pendiente",
        o => o.Cliente == "Constructora del Norte"
    };

    var ordenes = _unidad.Ordenes.ObtenerConPredicados(predicados);
    Console.WriteLine($"Órdenes: {ordenes.Count}");
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}");
    }
}
```

Línea 1: private void DemostrarPredicadosDinamicos() → declara el método.

Línea 3: Console.WriteLine("\n--- Predicados dinámicos ---"); → muestra la cabecera.

Línea 5: var predicados = new List<...> → crea la lista de predicados.

Línea 7: o => o.Estado == "Pendiente", → primer predicado.

Línea 8: o => o.Cliente == "Constructora del Norte" → segundo predicado.

Línea 9: }; → cierra la lista.

Línea 11: var ordenes = _unidad.Ordenes.ObtenerConPredicados(predicados); → llama al método del repositorio.

Línea 12: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 13: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 15: Console.WriteLine(\$" {orden.NumeroOrden} | {orden.Cliente} | {orden.Estado}"); → muestra los datos.

Paso 4: Llamar al método desde Ejecutar:

```
DemostrarPredicadosDinamicos();
```

Paso 5: Ejecutar dotnet run y verificar que las órdenes se muestran filtradas por los predicados dinámicos.

Resultado esperado: las órdenes pendientes de la "Constructora del Norte" se muestran correctamente.

Analogía final

La composición de consultas en una acería es como construir una orden de búsqueda al archivo central paso a paso. En lugar de pedir todos los documentos y después filtrarlos, el jefe de planta va añadiendo condiciones a la orden: primero el cliente, después el estado, después el rango de fechas. Cada condición se añade a la orden sin enviarla todavía. Solo cuando la orden está completa, el jefe la envía al archivo central. El archivo recibe la orden con todas las condiciones y devuelve solo los documentos que cumplen todas ellas. La ejecución diferida es como preparar la orden sin enviarla: se puede modificar hasta el último momento. La materialización prematura es como enviar la orden antes de terminarla: el archivo devuelve documentos que después hay que filtrar a mano. La composición de consultas permite construir consultas flexibles y eficientes, adaptadas a las necesidades de cada momento.

| Resultado esperado

Al final del ejercicio, deberías haber:

Creado el DTO FiltroOrdenesDto.

Añadido el método ObtenerConFiltros a la interfaz IOrdenRepositorio.

Implementado el método ConstruirConsultaConFiltros en OrdenRepositorio.

Creado el caso de uso ComposicionConsultasUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes.

Ejecutado las demostraciones de filtro por cliente, estado, rango de fechas, filtros combinados y ordenación con paginación.

Diagnosticado el error de materialización prematura.

Creado el método ObtenerConPredicados.

| Conexión con el siguiente punto

En este punto se ha profundizado en la composición de consultas y la ejecución diferida, incluyendo filtros condicionales, ordenaciones dinámicas, paginación y predicados dinámicos. Se ha comprobado que la composición permite construir consultas flexibles sin ejecutarlas hasta el final. En el siguiente punto se estudiarán las buenas prácticas en el acceso a datos, incluyendo la elección entre carga Eager, Lazy y Explicit, el uso de proyecciones y la composición de consultas.
